

Riverpod con anotaciones · Hoja de atajos

Sintaxis con `@riverpod` + `riverpod_generator` — instalación, providers y consumo · riverpod.dev

01 PUESTA EN MARCHA

1 Instalación

`pubspec.yaml`

POR LÍNEA DE COMANDOS

```
flutter pub add flutter_riverpod riverpod_annotation
flutter pub add dev:riverpod_generator
flutter pub add dev:riverpod_lint
flutter pub add dev:build_runner
```

RESULTADO EN PUBSPEC.YAML

```
dependencies:
  flutter_riverpod: ^3.0.0      # núcleo + widgets
  riverpod_annotation: ^4.0.0  # @riverpod
dev_dependencies:
  build_runner:      # ejecuta generadores
  riverpod_generator: # genera *.g.dart
  riverpod_lint:     # reglas de lint
  custom_lint:       # requerido por el lint
```

Variantes del paquete principal: `riverpod` (Dart puro), `flutter_riverpod` (Flutter) o `hooks_riverpod` (con flutter_hooks).

2 Activar el linter

`analysis_options.yaml`

```
analyzer:
  plugins:
    - custom_lint
```

`riverpod_lint` detecta errores comunes (providers mal declarados, dependencias faltantes) y agrega quick fixes y refactors en el IDE.

3 Generación de código

`build_runner`

```
# Una sola vez
dart run build_runner build -d
# Modo observador (regenera al guardar)
dart run build_runner watch -d
```

`-d` = `--delete-conflicting-outputs`: sobrescribe archivos generados en conflicto (casi siempre lo que quieres).

Cada archivo que declare providers anotados necesita la directiva `part` con el sufijo `.g.dart`:

```
import
'package:riverpod_annotation/riverpod_annotation.dart'
';
part 'mi_archivo.g.dart';
```

4 Configuración inicial

`main.dart`

Toda la app debe envolverse en un `ProviderScope`: ahí vive el estado de todos los providers.

```
void main() {
  runApp(const ProviderScope(child: MyApp()));
}
```

En Dart puro (sin Flutter) se usa `ProviderContainer`:

```
final container = ProviderContainer();
final value = container.read(helloWorldProvider);
```

5 Tu primer provider

```
@riverpod
String helloWorld(Ref ref) {
  return 'Hola mundo';
}
// El generador crea: helloWorldProvider
```

- Nombre generado = nombre de la función/clase + sufijo `Provider`.
- El generador elige el tipo de provider según el valor de retorno (sync, `Future`, `Stream`).
- Los providers son **perezosos**: no se ejecutan hasta que alguien los lee.

6 ¿Por qué anotaciones?

- Tipo deducido**: escribes tu lógica y Riverpod elige el provider correcto.
- Parámetros sin restricciones** (nombrados, opcionales, con valor por defecto) — sin el modificador `family`.
- Hot-reload con estado** del código de tus providers.
- Mejor depuración** gracias a metadatos generados.

¿SABÍAS QUE...?

Riverpod es un anagrama de «Provider». Nació para superar sus limitaciones: detección de errores en tiempo de compilación, varios providers del mismo tipo sin conflictos y lógica testeable sin necesidad de Flutter.

7 Funcional vs clase

Funcional: azúcar sintáctico de un provider de clase con solo `build()`; la UI no puede modificarlo. **Clase (Notifier):** expone métodos públicos para mutar el estado.

SÍNCRONO

```
@Riverpod
String example(Ref ref) => 'foo';

@Riverpod
class Example extends _$Example {
  @override
  String build() => 'foo';
  // métodos para mutar el estado...
  void update(String v) => state = v;
}
```

ASÍNCRONO – FUTURE

```
@Riverpod
Future<String> example(Ref ref) async {
  return Future.value('foo');
}

@Riverpod
class Example extends _$Example {
  @override
  Future<String> build() async => 'foo';
}
```

ASÍNCRONO – STREAM

```
@Riverpod
Stream<String> example(Ref ref) async* {
  yield 'foo';
}
```

Si la función es `async`, el provider gestiona automáticamente carga y errores, y expone un `AsyncValue<T>`.

TIP

Empieza siempre con una función y migra a clase (Notifier) solo cuando la UI necesite mutar el estado. El cambio es transparente para los consumidores: el nombre del provider generado se mantiene igual.

8 autoDispose y keepAlive

Con anotaciones, los providers son **autoDispose por defecto**: se destruyen cuando nadie los escucha.

```
// autoDispose (por defecto)
@riverpod
String example1(Ref ref) => 'foo';

// Mantener vivo siempre
@Riverpod(keepAlive: true)
String example2(Ref ref) => 'foo';
```

`@riverpod` (minúscula) usa valores por defecto; `@Riverpod(...)` (mayúscula) permite configurar opciones como `keepAlive`, `name` o `dependencies`.

9 Parámetros (adiós family)

La función del provider acepta cualquier número de parámetros: posicionales, nombrados, opcionales o con valor por defecto.

```
@Riverpod
Future<User> fetchUser(Ref ref,
  {required int userId}) async {
  final json = await http.get('api/user/$userId');
  return User.fromJson(json);
}

// Consumo:
ref.watch(fetchUserProvider(userId: 42));
```

Los parámetros deben tener un `=` consistente: cachea los valores o sobrescribe `=` (cada combinación de parámetros crea un estado independiente).

10 Equivalencias (manual → anotado)

SIN CODEGEN	CON @RIVERPOD
<code>Provider</code>	Función <code>sync</code>
<code>FutureProvider</code>	Función <code>Future</code>
<code>StreamProvider</code>	Función <code>Stream</code>
<code>NotifierProvider</code>	Clase + <code>build()</code> <code>sync</code>
<code>AsyncNotifierProvider</code>	Clase + <code>build()</code> <code>async</code>
<code>StreamNotifierProvider</code>	Clase + <code>build()</code> <code>stream</code>
<code>.family</code> / <code>.autoDispose</code>	Parámetros / por defecto

Actualmente `StateProvider`, `StateNotifierProvider` y `ChangeNotifierProvider` se consideran **legado** (disponibles vía import separado); el camino recomendado es `Notifier` / `AsyncNotifier`.

11 Consumir en widgets**OPCIÓN A — EXTENDER CONSUMERWIDGET**

```
class Home extends ConsumerWidget {
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final value = ref.watch(helloWorldProvider);
    return Text(value);
  }
}
```

OPCIÓN B — WIDGET CONSUMER PUNTUAL

```
Consumer(
  builder: (context, ref, _) {
    final tick = ref.watch(tickProvider);
    return Text('Tick: $tick');
  },
);
```

También existe `ConsumerStatefulWidget` + `ConsumerState` (con `ref` disponible como propiedad).

13 Combinar providers

Los providers también pueden hacer `ref.watch` de otros providers — la característica central de Riverpod:

```
@riverpod
bool isDivisibleBy4(Ref ref) {
  final tick = ref.watch(tickProvider);
  return tick % 4 == 0;
}
```

La UI que observe `isDivisibleBy4Provider` solo se reconstruye cuando el booleano cambia, no cada segundo.

Para reconstruir solo cuando cambia **una parte** del estado: `ref.watch(userProvider.select((u) => u.name))`.

TIP

Puedes testear providers sin UI: crea un `ProviderContainer` en el test y lee sus valores directamente — sin widgets, sin `pumpWidget` y sin mocks globales.

12 Ref: la API esencial

MÉTODO	USO
<code>ref.watch(p)</code>	Escucha declarativa. Reconstruye cuando <code>p</code> cambia. Tu opción por defecto en <code>build</code> .
<code>ref.read(p)</code>	Lectura puntual sin suscripción. Para callbacks (<code>onPressed</code> , etc.).
<code>ref.listen(p, cb)</code>	Escucha manual (<code>previous</code> , <code>next</code>) para efectos secundarios: diálogos, navegación, logs.
<code>ref.invalidate(p)</code>	Descarta el estado; se recalcula en la próxima lectura.
<code>ref.refresh(p)</code>	Atajo de <code>invalidate</code> + <code>read</code> : resetea y devuelve el nuevo estado.
<code>ref.onDispose(fn)</code>	Ciclo de vida: limpia recursos (timers, sockets) al destruirse el provider.

No uses `ref.read` para «optimizar» y evitar `ref.watch`: tu UI puede quedar desincronizada. *Watch en build, read en interacciones.*

`WidgetRef.listen` es seguro dentro de `build`. Fuera de `build` (p. ej. `initState`), usa `listenManual`.

14 AsyncValue

Los providers asíncronos exponen `AsyncValue<T>`: carga, dato o error en un solo tipo.

```
final user = ref.watch(fetchUserProvider(userId: 1));

// Con pattern matching (Dart 3):
return switch (user) {
  AsyncData(:final value) => Text(value.name),
  AsyncError(:final error) => Text('Error: $error'),
  _ => const CircularProgressIndicator(),
};

// O con .when:
user.when(
  data: (u) => Text(u.name),
  error: (e, st) => Text('$e'),
  loading: () => const CircularProgressIndicator(),
);
```

Getters útiles: `.isLoading`, `.hasError`, `.value`, `.valueOrNull` y `.requireValue`.

15 Notifiers: mutar estado

```
@riverpod
class Counter extends _$Counter {
  @override
  int build() => 0;
  void increment() => state++;
}

// En la UI:
ref.read(counterProvider.notifier).increment();
final count = ref.watch(counterProvider);
```

- `state`: lee/escribe el estado actual; notifica a los listeners al asignar.
- `provider.notifier`: accede a la instancia con los métodos públicos.
- Dentro de un Notifier, `this.ref` está disponible en cualquier parte.

17 Novedades clave

reciente

- **Ref unificado**: un solo tipo `Ref` (sin `FutureProviderRef`, `ExampleRef`, etc.).
- **Reintento automático**: si un provider falla al inicializar, reintenta con backoff exponencial (200 ms → 6.4 s). Configurable con `retry`.
- `@Riverpod(name: '...')`: define el nombre exacto del provider generado.
- **Mutations** (*experimental*): efectos secundarios con estados `isPending` / `hasError` / `isSuccess`; evita que `autoDispose` destruya el provider en plena operación.
- **Persistencia offline** (*experimental*): API `Storage` para cachear estado en disco (p. ej. SQLite).
- `invalidate(p, asReload: true)`: refresca conservando el valor previo como `AsyncValue` en recarga.
- **Pausa de providers**: los listeners invisibles se pausan automáticamente.

16 Overrides y ProviderScope

Reemplaza el comportamiento de un provider (entornos, inyección de valores iniciales, mocks):

```
ProviderScope(
  overrides: [

    apiUriProvider.overrideWithValue('https://dev.api'),
    repoProvider.overrideWith((ref) => FakeRepo()),
  ],
  child: MyApp(),
);
```

Un `ProviderScope` **anidado** permite hacer scoping: providers con estado distinto en una subparte del árbol de widgets.

18 ProviderObserver

Observa el ciclo de vida de todos los providers (logging, analítica):

```
class Logger extends ProviderObserver {
  @override
  void didUpdateProvider(context, prev, next) {
    print('${context.provider}: $next');
  }
}

ProviderScope(observers: [Logger()], child: MyApp());
```

19 Flujo de trabajo resumido

1. Instala dependencias y activa `custom_lint`.
2. Envuelve la app en `ProviderScope`.
3. Crea el archivo con `part 'x.g.dart'` y anota con `@riverpod`.
4. Corre `dart run build_runner watch -d`.
5. Consume con `ConsumerWidget` + `ref.watch`; muta con `.notifier`.

Docs: riverpod.dev/docs · Novedades: riverpod.dev/docs/whats_new

20 DO / DON'T rápidos

HAZ	EVITA
✓ <code>ref.watch</code> dentro de <code>build</code>	✗ <code>ref.read</code> dentro de <code>build</code>
✓ <code>ref.read</code> en callbacks/eventos	✗ <code>ref.watch</code> en callbacks
✓ Lógica de inicialización en <code>build()</code> del Notifier	✗ Inicializar estado en el constructor
✓ Providers como variables/funciones globales <code>final</code>	✗ Estado mutable global fuera de providers
✓ <code>ref.onDispose</code> para limpiar timers/streams	✗ Fugas de recursos al destruir providers
✓ Parámetros con <code>=</code> estable	✗ Pasar objetos nuevos en cada build como parámetro